

Git for Developers

Module 5 — Advanced Git Features

Rebase, Cherry-Pick, Stash, Tags, and Submodules

Module 5 — Agenda

Rewriting and Reorganizing History

- Rebasing: `git rebase` vs. `git merge`
- Interactive rebase: clean up commits before merging

Selective and Supplementary Tools

- Cherry-picking: apply individual commits across branches
- Stashing: save work-in-progress without committing
- Tags and releases: mark significant points in history
- Submodules: composing repositories from other repositories

Part 1 — Rebasing

git rebase vs. git merge

Both integrate changes from one branch into another. The result is different:

Merge (preserves branch topology):

```
main:      A ——— B ————— M
              \               /
feature:      C ——— D ——— E
```

Rebase (linear history):

```
main:      A ——— B ——— C' ——— D' ——— E'
```

Rebase rewrites history. Commits C, D, E become C', D', E' with

When to Rebase vs. Merge

Scenario	Preferred
Integrating a feature into <code>main</code> with merge commit	<code>merge --no-ff</code>
Keeping your feature branch up to date with <code>main</code>	<code>rebase</code>
Cleaning up messy commits before a PR	<code>rebase -i</code>
Shared branch (multiple contributors)	<code>merge</code> — never rebase
Personal feature branch (solo)	<code>rebase</code> is fine

git rebase — Basic Usage

Update your feature branch with the latest main commits

```
git checkout feature/user-auth
```

```
git rebase main
```

If conflicts occur during rebase:

1. Resolve the conflict in the file

2. git add <resolved-file>

3. git rebase --continue

Abort the rebase and return to original state

```
git rebase --abort
```

Rebase onto a specific branch instead of current

Part 2 — Interactive Rebase

`git rebase -i` — Rewriting History

Interactive rebase lets you edit, squash, reorder, or drop commits:

```
# Interactive rebase for the last 4 commits
```

```
git rebase -i HEAD~4
```

```
# Interactive rebase from a specific commit (non-inclusive)
```

```
git rebase -i a3f7c21
```

Opens your editor with a list of commits and actions.

Interactive Rebase Actions

```
pick a3f7c21 feat: add login endpoint
pick b2e1d40 wip: debug session handling
pick c9a8b12 fix typo
pick d7f3e56 feat: add logout endpoint
```

Commands:

p, pick = use commit as-is

r, reword = use commit, but edit the message

e, edit = use commit, but stop for amending

s, squash = meld into previous commit

f, fixup = like squash, but discard this commit's message

d, drop = remove commit entirely

Common Interactive Rebase Patterns

Squash WIP commits before merging

```
pick a3f7c21 feat: add login endpoint
squash b2e1d40 wip: debug session handling ← merged into above
squash c9a8b12 fix typo ← merged into above
pick d7f3e56 feat: add logout endpoint
```

Result: a clean 2-commit history instead of 4 messy commits.

Reorder commits

```
pick d7f3e56 feat: add logout endpoint      ← moved first  
pick a3f7c21 feat: add login endpoint
```

Drop a commit

```
drop c9a8b12 debug: log request body (contains secrets)
```

After interactive rebase, force-push to update the remote:

```
git push --force-with-lease
```

Part 3 — Cherry-Picking

`git cherry-pick` — Apply a Single Commit

Cherry-pick copies a commit from one branch and applies it to another:

```
# Apply a specific commit to the current branch
```

```
git cherry-pick a3f7c21
```

```
# Cherry-pick a range of commits
```

```
git cherry-pick a3f7c21..d7f3e56
```

```
# Cherry-pick without committing (stage only)
```

```
git cherry-pick --no-commit a3f7c21
```

```
# Cherry-pick and edit the message
```

```
git cherry-pick --edit a3f7c21
```

When to Use Cherry-Pick

Use case	Example
Backport a fix	Fix is on <code>main</code> , also needed on <code>release/v2.1</code>
Selective feature port	One commit from a big feature is needed now
Undo a wrong merge	Pick only the good commits from a bad merge
Hotfix	Fix on <code>hotfix</code> branch → apply to both <code>main</code> and <code>release</code>

Part 4 — Stashing

`git stash` — Shelve Work in Progress

Stash saves your uncommitted changes so you can switch context:

```
# Stash all modified tracked files and staged files
```

```
git stash
```

```
# Stash with a descriptive message
```

```
git stash push -m "WIP: half-done auth refactor"
```

```
# Stash including untracked files
```

```
git stash push --include-untracked
```

```
# List all stashes
```

```
git stash list
```

Applying and Managing Stashes

Apply the most recent stash (keeps it in the stash list)

```
git stash apply
```

Apply and remove from the stash list

```
git stash pop
```

Apply a specific stash

```
git stash apply stash@{2}
```

Delete a specific stash

```
git stash drop stash@{0}
```

Delete all stashes

Part 5 — Tags & Releases

`git tag` — Marking Releases

Tags are permanent references to specific commits — ideal for releases:

```
# Create a lightweight tag (just a pointer)
```

```
git tag v2.1.0
```

```
# Create an annotated tag (includes message, author, date — recommended)
```

```
git tag -a v2.1.0 -m "Release version 2.1.0 — add auth module"
```

```
# Tag a past commit
```

```
git tag -a v2.0.1 -m "Backport security fix" a3f7c21
```

```
# List tags
```

```
git tag
```

Lightweight vs. Annotated Tags

	Lightweight	Annotated
What it is	A pointer to a commit	A full Git object
Contains	Commit SHA only	Author, date, message, optional GPG signature
When to use	Local bookmarks, quick marks	Official releases (<code>git tag -a</code>)
Visible on GitHub	Yes	Yes, with extra metadata

Deleting and Checking Out Tags

Delete a local tag

```
git tag -d v2.0.0-beta
```

Delete a remote tag

```
git push origin --delete v2.0.0-beta
```

Check out code at a specific tag (detached HEAD)

```
git checkout v2.1.0
```

Create a branch from a tag (safer than detached HEAD)

```
git checkout -b hotfix/v2.1 v2.1.0
```

Part 6 — Submodules

`git submodule` — Repositories Within Repositories

A submodule embeds one Git repository inside another:

```
# Add a submodule  
git submodule add https://github.com/org/shared-lib.git libs/shared-lib
```

```
# After adding, this creates:  
# .gitmodules      (submodule config file, commit this)  
# libs/shared-lib/ (the checked-out submodule directory)
```

```
# .gitmodules  
[submodule "libs/shared-lib"]  
    path = libs/shared-lib  
    url  = https://github.com/org/shared-lib.git
```


Working with Submodules

Clone a repo with submodules – initialize and fetch them

```
git clone --recurse-submodules https://github.com/org/my-project.git
```

If you already cloned without --recurse-submodules

```
git submodule update --init --recursive
```

Update all submodules to the latest tracked commit

```
git submodule update --remote --merge
```

Check submodule status

```
git submodule status
```

Submodule Caveats

Consideration	Detail
Pinned to a commit	Submodule tracks a specific commit, not a branch tip
Two-step updates	Update the submodule, then commit the parent repo
Team discipline required	Everyone must run <code>--recurse-submodules</code> on clone
Complexity	Adds cognitive overhead — consider alternatives

Summary – Advanced Features

Command	What it does
<code>git rebase main</code>	Reapply branch commits on top of main
<code>git rebase -i HEAD~4</code>	Interactive rebase: squash, reorder, drop
<code>git cherry-pick <hash></code>	Apply a single commit to current branch
<code>git stash</code>	Shelve uncommitted changes
<code>git stash pop</code>	Restore most recent stash